# CompSci234/NetSys210 Advanced Topics in Networking
# Programming Assignment #1 (15%)
# Tiled 360 ° Video Streaming Using GPAC

Instructor: Cheng-Hsin Hsu
chsu@cs.nthu.edu.tw

University of California, Irvine
January 26, 2019

## 1 Objective

In this assignment, we will build a tiled-based DASH streaming system for 360° videos on Ubuntu Linux. The basic system architecture is shown in Figure 1. The **streaming server** (Apache) stores MPD and segment files at different quality levels - low, mid, and high. These files can be downloaded through HTTP GET requests sent from the **streaming client**. The **streaming client** (GPAC MP4Client) performs adaptive streaming according to the network conditions. To optimize the viewing experience, the **streaming client** automatically selects the highest quality level it can afford of for individual tiled-segments.

As a starting point, you are given a running **streaming server** and a basic framework of the **streaming client** adapted from GPAC MP4Client. The **streaming server** contains the MPD file and the encoded tiled-video segments. The **streaming server** is essentially a local Apache2 server and the files are accessible through `http://localhost/shark/`. As for the **streaming client**, we provide a modified GPAC MP4Client for you to work on. The **streaming server** and **client** are bundled in a VM image prepared by us, to save you some time from setting up all the dependencies.

Your job is to modify the MP4Client to realize tiled-based DASH streaming. This is done in two steps. First, you parse a MPD file to extract metadata from it. Next, you adjust the tile quality levels based on the parsed metadata. In the process, you will also sharpen your C++ programming skills in a mid-size project.

## 2 Descriptions

Dynamic Streaming over HTTP (DASH) is a popular multimedia streaming approach. It is pull-based and thus is different from the traditional push-based approaches like RTSP/RTP/RTCP streaming. Compared to other proprietary pull-based solutions, such as Apple HLS, Adobe HDS, and Microsoft Smooth Streaming, DASH streaming is standardized and not locked in by any vendor. To support DASH, the server is a simple HTTP web server (e.g., Apache). The media (video) is divided into small individually decodable/playable video segments which are, for example, 10 seconds long. These segments are also called streamlets. The client media player retrieves the streamlets from the web server, one at a time, and plays them while downloading future segments. For the client to know the streamlet files that belong to a video, the server provides a playlist file in Media Presentation Description (MPD) format. MPD is basically an XML file defined in the MPEG DASH standard. To start streaming, the client loads the MPD playlist file and then selectively downloads the streamlets that are listed in this file.

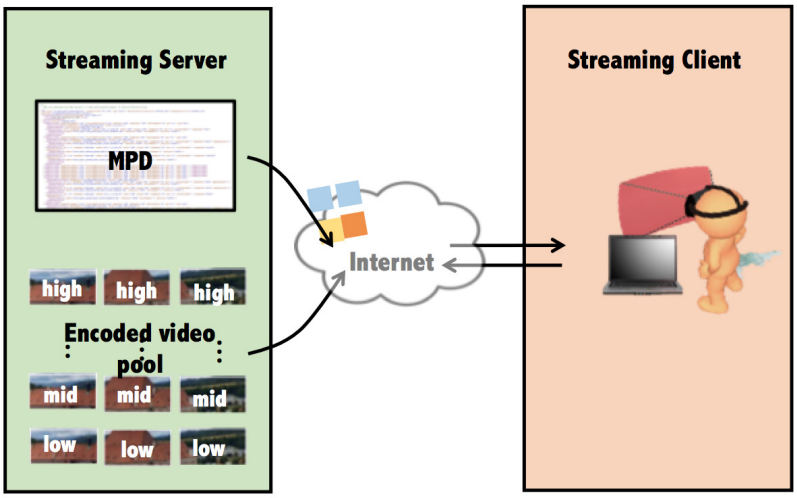


Figure 1: Architecture of our streaming system.

# 3 Software Tools

We have prepared a VM at `https://tinyurl.com/y7tf398q`. You may use VirtualBox to import the image; while using other hypervisors is also possible. The username is test and the password is compsci234. Once you login to the VM, you should see a gpac/ folder under your home directory. Please use that folder as the starting point of this assignment.

## 3.1 GPAC MP4Client

### 3.1.1 Description

GPAC [1] is an Open Source cross-platform multimedia framework being developed at Telecom ParisTech (a.k.a. ENST). It covers different aspects of multimedia supports, with a focus on presentation technologies (graphics, animation, and interactivity) and on container formats such as MP4. GPAC provides three sets of tools: multimedia player, called Osmo4/MP4Client [2], a multimedia packager, called MP4Box, and some server tools, such as MP4Box and MP42TS. We use MP4Client as our player for 360° video streaming.

### 3.1.2 Installation

GPAC has been pre-installed in the VM image. If you need to install GPAC in the future, please refer to their website [3] for more details.

### 3.1.3 Sample Usage

Basic usages are introduced below, for more options, please refer to the man page [4].

1. Recompile and reinstall the project every time you make modifications:
   $ cd ~/gpac
   under gpac:
   $ make
   $ sudo make install (password: compsci234)

2. Video playing:
    - Basic one with log
      $ MP4Client -gui `http://localhost/shark/dash_shark.mpd`
    - Using GPAC log system [5] to keep track of what's going on
      $ MP4Client -gui `http://localhost/shark/dash_shark.mpd` -logs dash@debug:container:sync@error

# 4 Tasks

This assignment contains two tasks:

- *Task 1*: Find out where MP4Client parses the MPD file. Make some modifications to enable it to parse "model" in each AdaptationSet and print it in the terminal.
- *Task 2*: Assign individual tiles diverse quality levels based on the "model" values from the MPD file.

Upon completing the two steps, you'll be able to watch a tiled 360° video streamed by DASH. To bootstrap your work, the functions you are likely to modify are marked with comments: [nmsl_TODO&HINT]. Please look for it in the source code. Please read the code carefully to make sure that you understand how the system works.

## 4.1 Task 1: Find out Where Mp4Client Parses "Model"

### 4.1.1 Purpose

To get familiar with the MPD structure and see how DASH uses MPD.

### 4.1.2 Subtasks

MPD is a hierarchical data model written in the XML file format. It points to different quality levels of the media content available to clients. Therefore, a playback client adaptively streams the tile representations that fit its bandwidth the most based on the metadata parsed from the MPD. In the following paragraphs, we will walk you through the structure of MPD. We will show you how it is used for video streaming, and explain the tasks you need to complete.

1. **The MPD hierarchy**
    - *Period*: Period is the top level element in a MPD. It describes a part of the content given a start time and duration. One or more periods can be included in a single MPD. In this assignment, we consider a single period that lasts for 1 minute.
    - *AdaptationSet*: Each period contains one or more AdaptationSets that enable the logical grouping of different multimedia components. For example, multimedia components with the same codec, language, resolution, audio channel format, etc. could be put within the same AdaptationSet. In this assignment, we use AdaptationSet to bundle representations of the same tile together, which are essentially the same tile in different quality levels (and thus bitrates). Currently, MP4Client is able to parse "bitstreamSwitching", "lang", "maxFrameRate", "maxHeight", "maxWidth", "par", and "segmentAlignment" in an AdaptationSet. Your task is to parse "model" and print it to the standard out in the format specified below.
    - *Representation*: Each Representation indicates a version of a given video content/sequence. This multi-representation strategy gives clients the possibility to switch among versions. The client can therefore adapt the media stream to the current network conditions, in order to guarantee smooth playback. In this assignment, we have three Representations in an AdaptationSet, low, mid, and high, respectively. Each Representation points to a tile segment at a specific bitrate.
2. **Goal: Parse "model" yourself and print it in the terminal in the following form**:
   `adaptation_set_num: 1, model: 1.`

## 4.2 Task 2: Assign Tile Quality Levels Based on Their "Model" Values

### 4.2.1 Purpose

Understand how tile-based DASH can be realized.

### 4.2.2 Subtasks

1. **Segment and Tile.** To realize DASH, we chop the media file into segments along the time domain, e.g., 4 sec for each segment. The segments are then encoded at different bitrates or spatial resolutions. For each segment, we further spatially divided it into several tiles, e.g., 3x3 or 5x5.

2. **Locate where the MP4Client assigns representation to each tile and set it with the "model" value.** We have three representations for each tile, i.e., **low**, **mid**, and **high**, and we use **0**, **1**, and **2** as the indexes. Right now, the representation index of each tiles is set to 0. Therefore, the lowest-quality tiles are always requested. In this task, you'll assign the tile representation indexes with the values you get from the "model" field of AdaptationSet. Take Figure 2 as an example, tiles (left to right, then top to bottom) have the "models": 1, 0, 0, 0, 2, 0, 1, 0, 0. Hence, during playout, the video quality of individual tiles are: **mid**, **low**, **low**, **low**, **high**, **low**, **mid**, **low**, **low**. The outcome of this assignment is a video playout with tiles in diverse quality levels.


Figure 2: Example of tiled video with diverse "models".

# 5 Submission Guidelines

The grades are given mainly based on your report. If your report fails to convince the instructor, you may be asked to give a live demo.

## 5.1 Report

Following questions need to be answered in your report. please write them into four sections.

1. Briefly explain how you implement Task 1. Please include the code snippets showing your modifications.
2. Can the current MP4Client handle multiple Periods in an MPD file? If not, how would you modify it to support that? Please include code snippet to elaborate.
3. Briefly explain how you implement Task 2. Please include the code snippets showing your modifications.

4. In task 2, how many times is dash_do_rate_adaptation called for downloading each segment? What if we switch the video into 5x5 tiling, how many times would it be?

Please submit your report in PDF and PDF only.

### 5.2 Late Submission Policy

- Late within 24 hours: 25% reduction in marks.
- Late within a week: 50% reduction in marks.
- After one week: your submission will not be graded.

## 6 Grading Policy

- Task 1 (5 pts): Successfully print out the "model" values in the terminal.
- Task 2 (4 pts): Successfully play out the tiled video with intended (diverse) representations.
- Report (6 pts): Answers to questions 2 and 4.

## 7 Additional Notes

- For more details on MPD, you may check out
  `https://bitmovin.com/dynamic-adaptive-streaming-http-mpeg-dash/`
  `https://www.brendanlong.com/the-structure-of-an-mpeg-dash-mpd.html`
- To get to know GPAC more, please check its doxygen page
  `http://download.tsi.telecom-paristech.fr/gpac/doc/gpac/index.html`

## References


[1] `https://gpac.wp.imt.fr/home/.`

[2] `https://gpac.wp.imt.fr/player/.`

[3] `https://gpac.wp.imt.fr/downloads/.`

[4] `http://manpages.ubuntu.com/manpages/cosmic/man1/MP4Client.1.html.`

[5] `https://gpac.wp.imt.fr/2011/08/04/recent-evolutions-in-the-gpac-log-system/.`